T.I.P. TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES — COMPUTER ENGINEERING — QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structure Algorithms	**SECTION:**	CpE21S2
DATE PERFORMED:	9/18/2026	**DATE SUBMITTED:**	9/26/2026
NAME:	Esguerra, Paula	**INSTRUCTOR:**	Engr. Jimlord Quejado

ASSIGNMENT 9.1 Binary Search Tree

1. PROCEDURE OUTPUT

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Step 1:

Code Screenshot

__main__

```cpp
#include <iostream>
#include "BST_2.h"

int main(){
    //create the root node:
    GeneralTree* a = new GeneralTree('A');
    GeneralTree* b = new GeneralTree('B');
    GeneralTree* c = new GeneralTree('C');
    GeneralTree* d = new GeneralTree('D');
    GeneralTree* e = new GeneralTree('E');
    GeneralTree* f = new GeneralTree('F');
    GeneralTree* g = new GeneralTree('G');
    GeneralTree* h = new GeneralTree('H');
    GeneralTree* i = new GeneralTree('I');
    GeneralTree* j = new GeneralTree('J');
    GeneralTree* k = new GeneralTree('K');
    GeneralTree* l = new GeneralTree('L');
    GeneralTree* m = new GeneralTree('M');
    GeneralTree* n = new GeneralTree('N');
    GeneralTree* p = new GeneralTree('P');
    GeneralTree* q = new GeneralTree('Q');

    //Connecting node children to root a
    a -> addChild(b);
    a -> addChild(c);
    a -> addChild(d);
    a -> addChild(e);
    a -> addChild(f);
    a -> addChild(g);
```

```
31      //connecting children to their parents
32      d -> addChild(h);
33
34      e -> addChild(i);
35      e -> addChild(j);
36
37      f -> addChild(k);
38      f -> addChild(l);
39      f -> addChild(m);
40
41      g -> addChild(n);
42
43      // connecting children to j
44      j -> addChild(p);
45      j -> addChild(q);
46
47
48      std::cout << "General Tree preOrder Traversal: ";
49      a->preOrder();
50      std::cout << std::endl;
51      std::cout << std::endl;
52
53  }
```

__header__

```
#ifndef GENERAL_TREE_H
#define GENERAL_TREE_H

#include <iostream>

class GeneralTree {
private:
    char key;
    GeneralTree* firstChild;
    GeneralTree* next;

public:

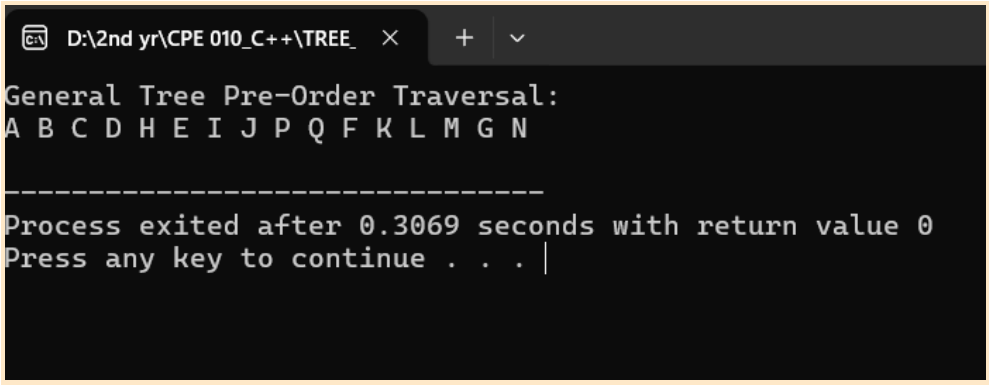
    GeneralTree(char value) {
        key = value;
        firstChild = nullptr;
        next = nullptr;
    }


    void addChild(GeneralTree* child) {
        if (firstChild == nullptr) {
            firstChild = child;
        } else {

            GeneralTree* temp = firstChild;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = child;
        }
    }


    void preOrder() {
        std::cout << key << " ";
        if (firstChild != nullptr) {
            firstChild->preOrder();
        }
        if (next != nullptr) {
            next->preOrder();
        }
    }
};

#endif
```

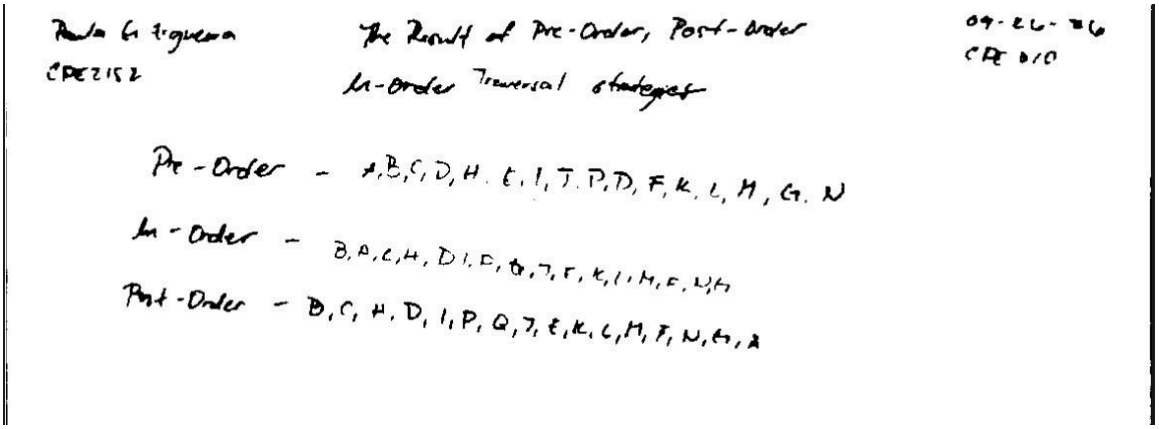
Output	
Comments	The key is the one that stores the value of the node. The children is the one that holds memory pointers to the other nodes where like node A is a vector that stores pointer of B, C, D, E, F and G. where it different from which order it will print. .In preOrder it will follow, Root->Left->Right. In inOrder it will follow Left→Root->Right. And lastly on preOrder, Left->Right->Root. As you can see, it differs where and when it will print root.
Step 2:	

Node	Height	Depth
A	3	0
B	0	1
C	0	1
D	1	1
E	2	1
F	1	1
G	1	1
H	0	2
I	0	2
J	1	2
K	0	2

L	0	2
M	0	2
N	0	2
P	0	3
Q	0	3

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post- order traversal.

Tree Diagram



Function	

preOrder	__main__
	`std::cout << "++++++++++++++++++++++++++++++++++++++++++++++++++++++++\n";`
	`std::cout << "This is my Output:\n\n";`
	`std::cout << "preOrder: ";`
	`a->preOrder();`
	`std::cout << std::endl;`
	
	__header__
	`// Pre-order traversal to verify structural accuracy`
	`void preOrder() {`
	`std::cout << key << " ";`
	`for (GeneralTree* child : children) {`
	`child->preOrder();`
	`}`
	`}`

Output: preOrder	D:\2nd yr\CPE 010_C++\TREE_ ++ This is my Output: preOrder: A B C D H E I J P Q F K L M G N inOrder: B A C H D I E P J Q K F L M N G postOrder: B C H D I P Q J E K L M F N G A ++ Finding Data: J was FOUND! M was FOUND! ++ Searching for new node 'O' using Pre-order: O was FOUND! ++ -------------------------------- Process exited after 0.4189 seconds with return value 0 Press any key to continue . . .

inOrder

__main__

```
56      std::cout << "inOrder: ";
57      a->inOrder();
58      std::cout << std::endl;
59
60
```

__header__

```
40      void inOrder() {
41          if (children.empty()) {
42              std::cout << key << " ";
43              return;
44          }
45
46          children[0]->inOrder();
47          std::cout << key << " ";
48          for(size_t i = 1; i < children.size(); ++i){
49              children[i]->inOrder();
50          }
51      }
```

Output:
inOrder

```
D:\2nd yr\CPE 010_C++\TREE_
++++++++++++++++++++++++++++++++++++++++++++++++++++++++
This is my Output:

preOrder: A B C D H E I J P Q F K L M G N
inOrder: B A C H D I E P J Q K F L M N G
postOrder: B C H D I P Q J E K L M F N G A

++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Finding Data:
J was FOUND!
M was FOUND!

++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Searching for new node 'O' using Pre-order:
O was FOUND!
++++++++++++++++++++++++++++++++++++++++++++++++++++++++

--------------------------------
Process exited after 0.4189 seconds with return value 0
Press any key to continue . . .
```

postOrder

__main__

```
60
61          std::cout << "postOrder: ";
62          a->postOrder();
63          std::cout << std::endl;
64          std::cout << std::endl;
65
```

__header__

```
33          void postOrder() {
34              for (GeneralTree* child : children) {
35                  child->postOrder(); //  FIXED: Changed from preOrder() to postOrder()
36              }
37              std::cout << key << " ";
38          }
39
```

Output:
postOrder

```
D:\2nd yr\CPE 010_C++\TREE_
++++++++++++++++++++++++++++++++++++++++++++++++++++++++
This is my Output:

preOrder: A B C D H E I J P Q F K L M G N
inOrder: B A C H D I E P J Q K F L M N G
postOrder: B C H D I P Q J E K L M F N G A

++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Finding Data:
J was FOUND!
M was FOUND!

++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Searching for new node 'O' using Pre-order:
O was FOUND!
++++++++++++++++++++++++++++++++++++++++++++++++++++++++

--------------------------------
Process exited after 0.4189 seconds with return value 0
Press any key to continue . . .
```

Comments:

The program matches the first output to the second output of the functions, where the relational integrity of the parent-child remains accurate. The outputs of both functions are “A B C D H E I J P Q F K L M G N”. I haven’t put the output of inorder and postorder in the first function, but if I did, it have the same output because the logic will be implemented there as the very similar to the second function. Where postorder will have “B C H D I P Q J E K L M F N G A” and inorder will have “B A C H D I E P J Q K F L M N G ”.

Function: findData

__main__

```
65
66      std::cout << "+++++++++++++++++++++++++++++++++++++++++++++++++++++++++\n";
67      std::cout << "Finding Data:\n";
68      a->findData("PRE", 'J');
69      a->findData("IN", 'M');
70      std::cout << std::endl;
71
72      std::cout << "+++++++++++++++++++++++++++++++++++++++++++++++++++++++++\n";
73      //adding "O"
74      GeneralTree* o = new GeneralTree('O');
75      g->addChild(o);
76
77      std::cout << "Searching for new node 'O' using Pre-order:\n";
78      a->findData("PRE", 'O');
79      std::cout << "+++++++++++++++++++++++++++++++++++++++++++++++++++++++++\n";
80
81      return 0;
82
```

__header__

```
52
53      bool SearchFunction(const std::string& choice, char targetKey, bool& foundFlag){
54          if (choice == "PRE" && !foundFlag) { // Match main.cpp choice strings
55              if (key == targetKey) {
56                  foundFlag = true;
57                  std::cout << key << " was FOUND!" << std::endl;
58                  return true;
59              }
60          }
61
62          if (choice == "IN" && !children.empty()){
63              children[0]->SearchFunction(choice, targetKey, foundFlag);
64          }
65
66          if (choice == "IN" && !foundFlag && key == targetKey){
67              foundFlag = true;
68              std::cout << key << " was FOUND!" << std::endl;
69              return true;
70          }
71
72          //  FIXED: Corrected quote positioning for the ternary operator
73          for (size_t i = (choice == "IN" ? 1 : 0); i < children.size(); ++i){
74              children[i]->SearchFunction(choice, targetKey, foundFlag);
75          }
76
77          if(choice == "POST" && !foundFlag && key == targetKey){
```

```
78                  foundFlag = true;
79                  std::cout << key << " was FOUND!" << std::endl;
80                  return true;
81              }
82              return foundFlag;
83          }
84
85          void findData(const std::string& choice, char targetKey){
86              bool foundFlag = false;
87              SearchFunction(choice, targetKey, foundFlag);
88          }
89  };
90
91  #endif
92
```

Output:
findData

```
D:\2nd yr\CPE 010_C++\TREE_
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
This is my Output:

preOrder: A B C D H E I J P Q F K L M G N
inOrder: B A C H D I E P J Q K F L M N G
postOrder: B C H D I P Q J E K L M F N G A

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Finding Data:
J was FOUND!
M was FOUND!

++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Searching for new node 'O' using Pre-order:
O was FOUND!
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++

--------------------------------
Process exited after 0.4189 seconds with return value 0
Press any key to continue . . .
```

Comments:

Adding the node “o” under the parent letter of G adjusts the size of the letter G’s child array, since the function utilizes pointers transversal, findData can successfully locates the new node without needing to restructure or rearrange the system.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1:

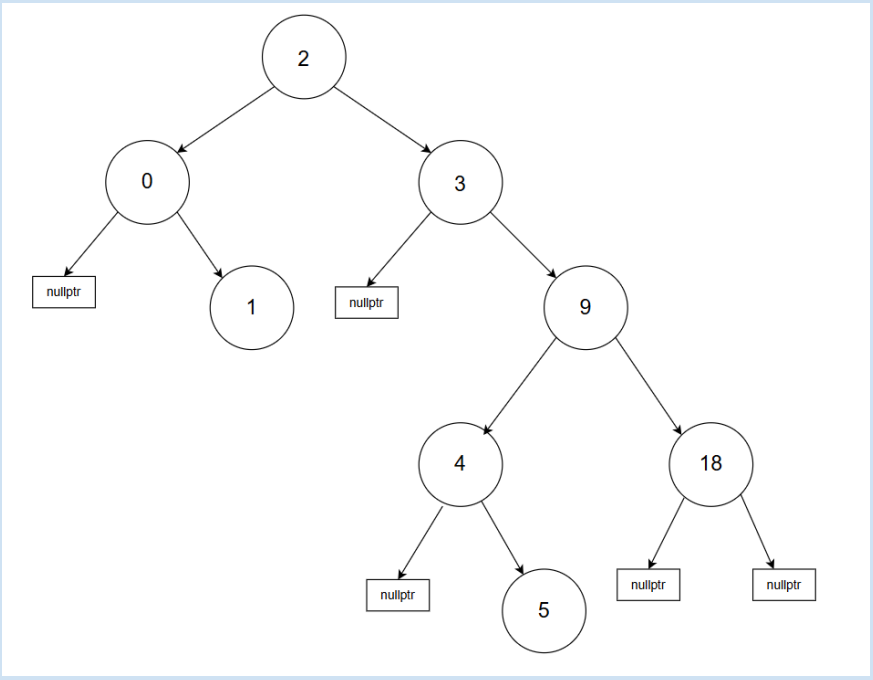
```
#include <iostream>
#include "BST_2.h"

int main(){
    //create the root node:
    BinarySearchTree<int> tree(20);

    tree.insert(2);
    tree.insert(3);
    tree.insert(9);
    tree.insert(18);
    tree.insert(0);
    tree.insert(1);
    tree.insert(4);
    tree.insert(5);
```

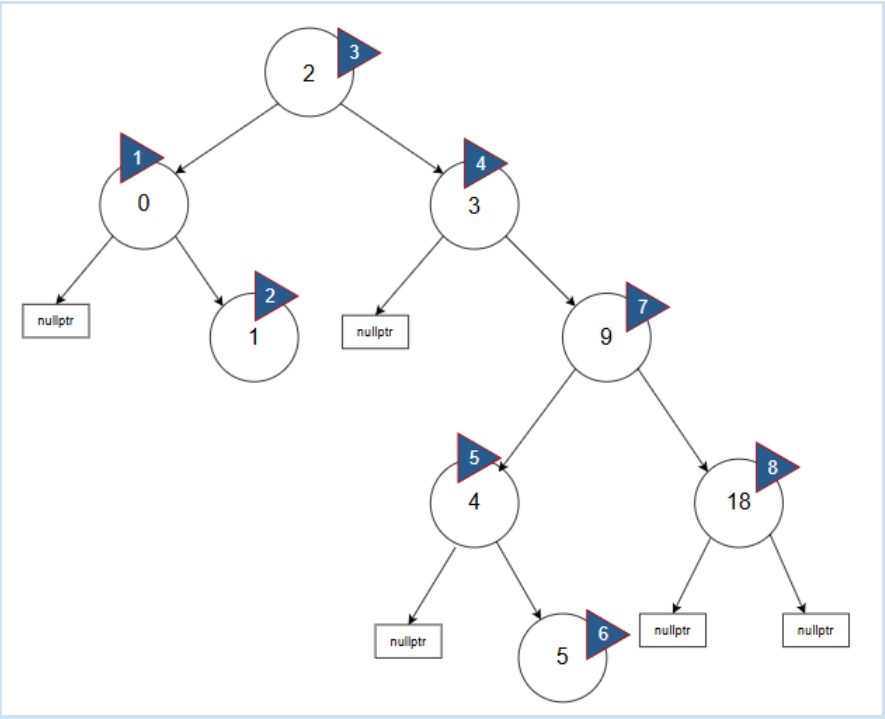
Step 2:

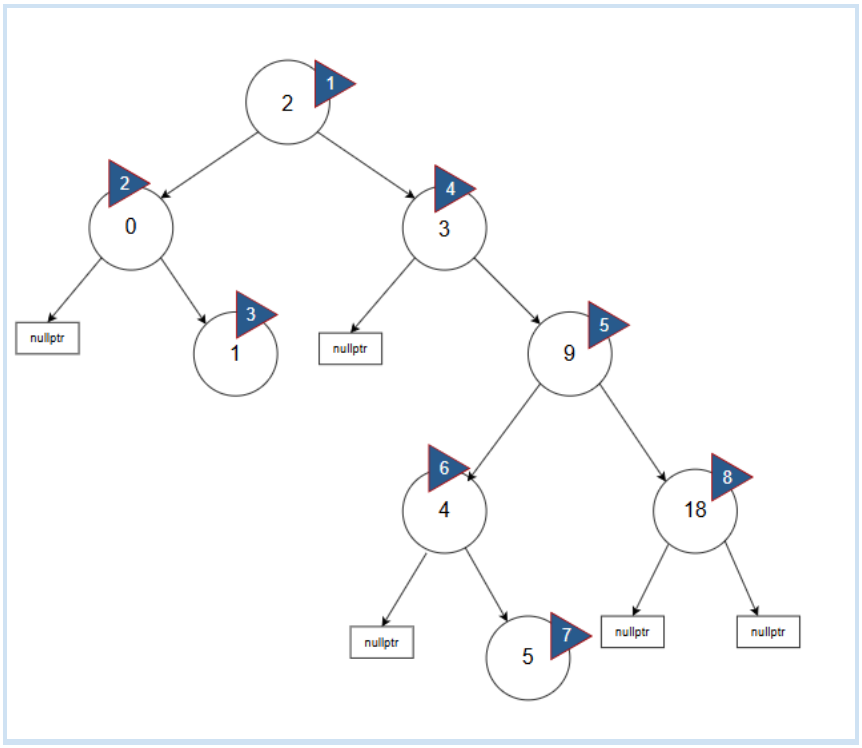
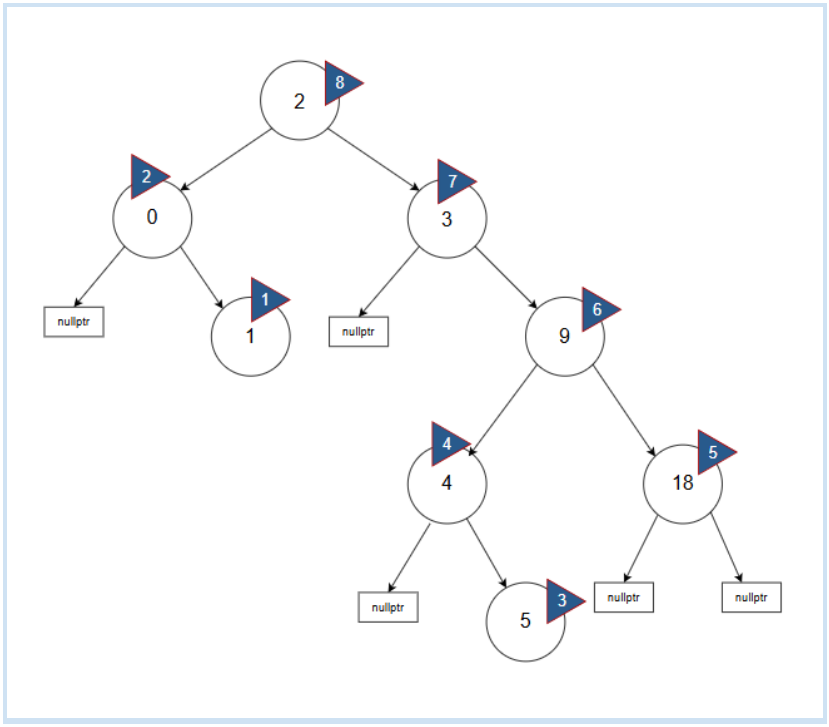
Tree Diagram

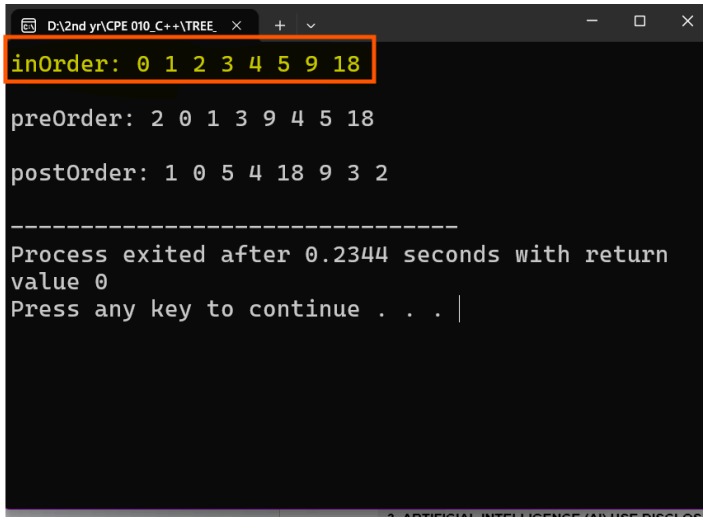
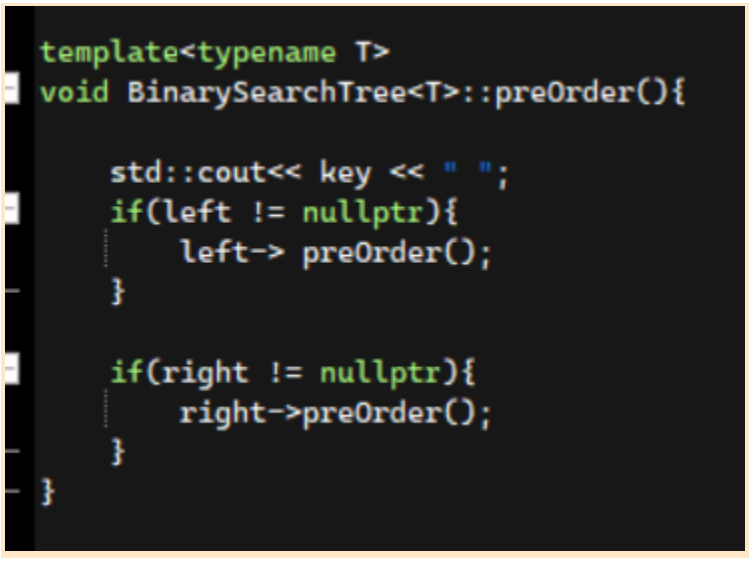


InOrder

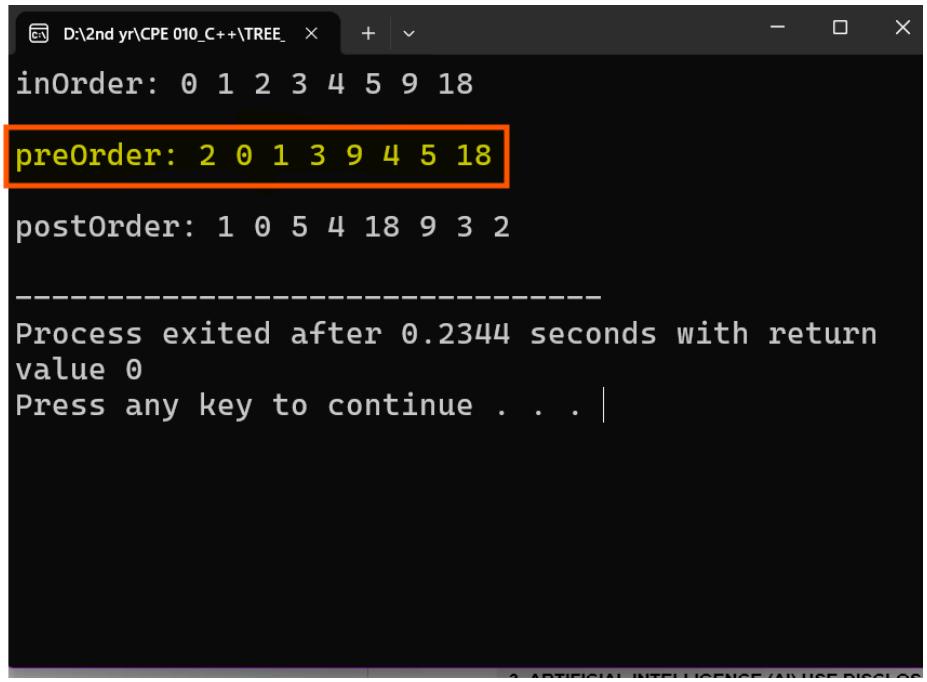
Left -> Root -> Right



PreOrder Root -> Left -> Right	
PostOrder Left -> Right -> Root	

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?	
inOrder	The inOrder traversal was performed using the inOrder() member function of the BinarySearchTree class. 
preOrder	__Function__ 

__Output__

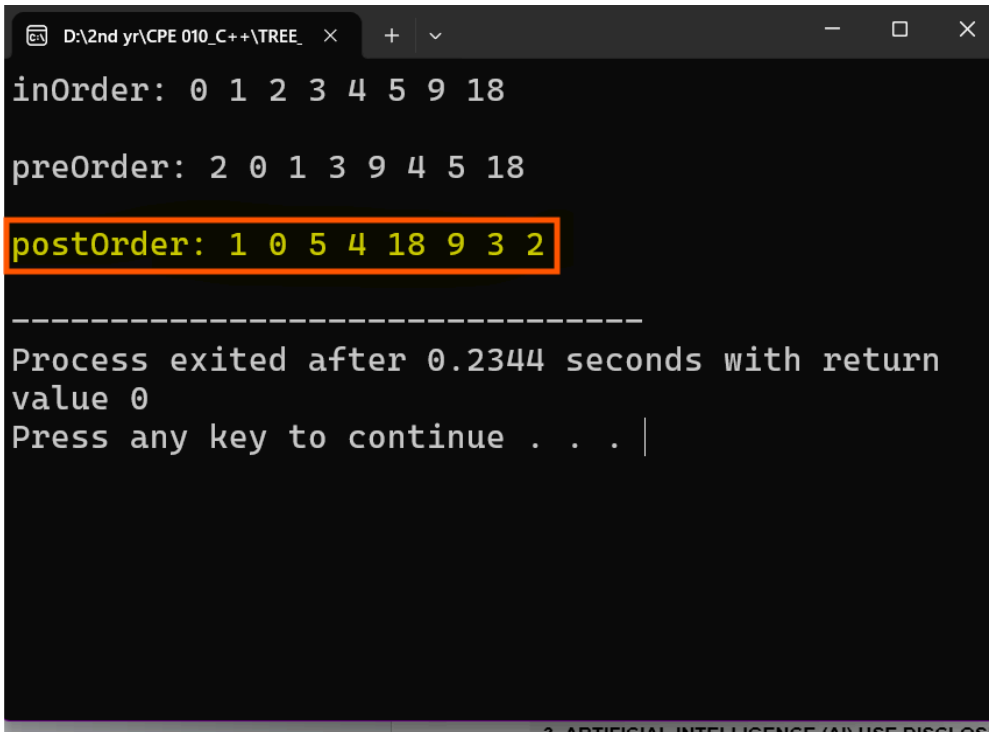


postOrder

__Function__

```
template<typename T>
void BinarySearchTree<T>::postOrder(){
    //go to left
    if(left != nullptr){
        left->postOrder();
    }

    //go to right
    if(right != nullptr){
        right->postOrder();
    }
    std::cout << key << " ";
}
```

	__Output__ 
Answer to Step 3 question:	The inOrder Tranversal performed the function of the BinarySearchTree class. The output have match the paths that are indicated to them successfully. They are identical to the step by step numbering sequences of the diagram that I have created based on the given.

Conclusion:
I have fulfilled the requirements that need to be demonstrated and implemented to the application of general trees and binary search tree using c++. The program built a general tree and BST (binary search tree) by following the hierarchy structure. BST elements were accorded to the rule, where the smaller values are placed on the left and the biggest to the right. Using the implementation of the three transversal algoritms, which are, Pre-Order, In-Order, and Post-Order, the terminal console successfully matched the different type of tracing sequences. And lastly, the implementation of findData function shows that the pointer-based transversal are locate by target keys, in specific nodes or added ones, without rearranging the system of the code. In summary, I have learned the different structures provided for hierarchical data representation, organizing datta and data retrieval.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea Generation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I have used it for the purpose of having ideas and expanding the ideas I want to learn,**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.